

Campus Energy Meter Data Processing and Analysis



***SUBJECT: PROGRAMMING FOR PROBLEM
SOLVING USING PYTHON***

COURSE CODE: ETCCPP102

MADE BY: Anshuman Sharma

CLASS: B.TECH CSE (AI ML)

SECTION: B

ROLL NO: 2501730313

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to my project guide, Sameer Farooq, for their constant guidance, motivation, and support throughout the completion of this project. Their valuable insights and encouragement helped me stay focused and improve my work at every stage.

I am also sincerely thankful to the Head of Department and all the faculty members for providing the necessary facilities, resources, and a positive environment that made this project possible. Their support and cooperation have been truly appreciated.

Lastly, I would like to thank my friends and family for always being there for me. Their help, patience, and moral support kept me going, especially during the challenging parts of this journey.

TABLE OF CONTENT

1. Introduction
2. Problem statement & objectives
3. Methodology and System Design
4. Implementation Details
5. Results and Screenshots
6. Discussion and Analysis
7. Advantages and application
8. Limitation and future scope
9. Reflection
10. Conclusion
11. References

INTRODUCTION

Energy consumption in educational campuses is increasing due to the growing number of buildings, laboratories, computer centers, and other facilities. Without proper monitoring, a lot of electricity gets wasted, which leads to higher bills and negative impact on the environment. Therefore, it is important to analyze energy usage data and identify patterns, peaks, and high consuming buildings so that the campus can plan energy-saving strategies.

In this project, a Python-based data analysis pipeline is developed to study campus electricity consumption using meter reading data stored in CSV files. The project uses Pandas for data handling, Matplotlib for visualizations, and object-oriented programming for modeling buildings and meter readings. The system reads multiple CSV files from a folder, cleans and combines them, performs daily and weekly aggregations, calculates building-wise summaries, generates graphs, and exports clean data and summary reports.

Overall, this project helped to learn how to work with real-world style data, handle timestamps, perform resampling, and convert raw energy readings into meaningful insights. It also shows how Python can be used in practical scenarios related to energy management and smart campus systems

PROBLEM STATEMENT & OBJECTIVES

PROBLEM STATEMENT

Campus authorities usually receive monthly electricity bills but they do not always have a detailed breakdown of which building is consuming how much energy and on which days the usage is highest. Manual analysis of large CSV meter files is time-consuming and error-prone. There is a need for an automated script that can read multiple data files, clean them, summarize the data, and present it in a simple visual form that anyone can understand.

OBJECTIVES

The main objectives of the project are:

- ✓ To build a Python script that automatically reads all CSV meter reading files from a given “data” folder.
- ✓ To clean and validate the timestamp column and remove invalid rows.
- ✓ To calculate daily and weekly total energy consumption using Pandas resampling.
- ✓ To generate a building-wise summary including mean, minimum, maximum, and total kWh for each building.

- ✓ To model buildings and meter readings using object-oriented programming for better structure and flexibility.
- ✓ To create a visualization dashboard with daily trend line, weekly bar chart, and scatter plot of all readings.
- ✓ To export cleaned data and summary information to CSV and text files for further use.

Each of these objectives is implemented step by step in the code in detail in this report.

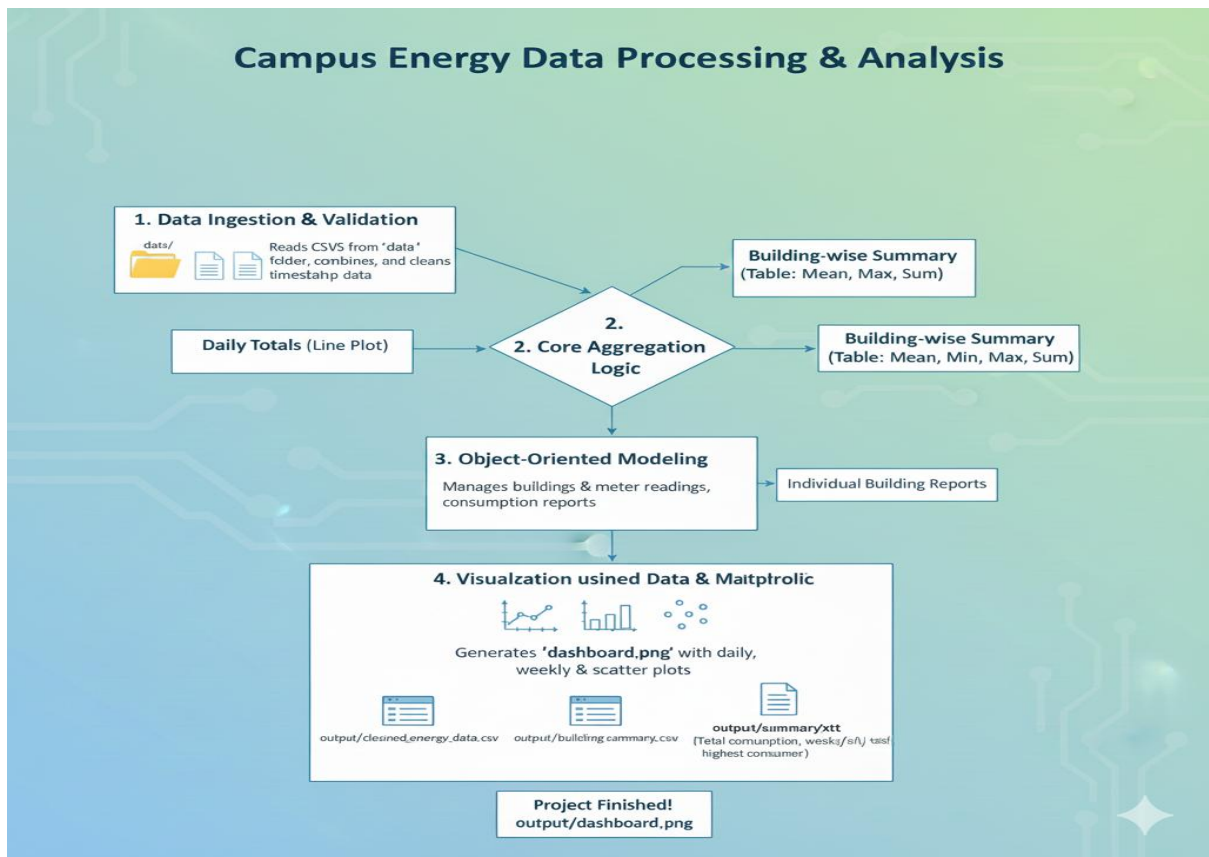
Methodology and System Design

Overflow Workflow

The overall workflow of the program is explained below:

1. Read all CSV files from the data folder.
2. Add a "source_file" column to each DataFrame and merge them.
3. Convert the timestamp column to datetime and drop invalid entries.
4. Compute daily and weekly totals and building-wise statistics.
5. Use OOP classes (MeterReading, Building, BuildingManager) to represent data logically.
6. Generate visualizations using Matplotlib.
7. Export clean data and summaries to the output folder.

This workflow make sure that every important task from ingestion to reporting is covered in a structured manner.



Data Flow

- Input: Multiple CSV files → combined DataFrame df
- Processing:
 - Cleaning timestamps
 - Resampling and grouping
 - Creating objects and calculating totals
- Output:
 - dashboard.png
 - cleaned_energy_data.csv
 - building_summary.csv
 - summary.txt

Each output has a specific purpose: dashboards are for visual analysis, CSV exports for further tools, and summary.txt for a quick textual overview.

IMPLEMENTATION DETAILS

At the beginning of the code, necessary libraries are imported: pandas as pd, matplotlib.pyplot as plt, os, and Path from pathlib. The script prints the working directory and creates an “output” folder using `os.makedirs("output", exist_ok=True)` so that export operations do not fail.

Task 1: Data Ingestion and Validation

The data folder is set as `data_folder = "data"` and all CSV files are listed with `Path(data_folder).glob("*.csv")`. For each file in the list, `pd.read_csv(file)` is used to load the data into a DataFrame, and a new column “source_file” is added with `file.name` to remember from which file each row came. These DataFrames are stored in a list called `combined_data`.

If any file causes an error during reading, the exception is caught and printed, so that one bad file does not stop the whole program. After processing all files, the script checks if `combined_data` is empty; if yes, it prints “No data found.” and exits. Otherwise, it concatenates all the DataFrames using `pd.concat(combined_data, ignore_index=True)` to produce a single DataFrame `df`.

The timestamp column is then converted into datetime format using `pd.to_datetime(df["timestamp"], errors="coerce")`, which converts invalid timestamps to NaT (Not a Time). Rows where the timestamp is NaT are dropped using `df.dropna(subset=["timestamp"])`. Finally, `df.head()` is printed to show a preview of the loaded and cleaned data.

Task 2: Core Aggregation Logic

Three functions are defined to handle the main aggregations: `calculate_daily_totals`, `calculate_weekly_totals`, and `building_wise_summary`.

- `calculate_daily_totals(data)` uses `data.resample("D", on="timestamp")["kwh"].sum()` to compute total kWh per day.
- `calculate_weekly_totals(data)` uses `data.resample("W", on="timestamp")["kwh"].sum()` to compute weekly totals.
- `building_wise_summary(data)` uses `data.groupby("building")["kwh"].agg(["mean", "min", "max", "sum"])` to produce detailed statistics per building.

The script calls these functions as follows:

- `daily_totals = calculate_daily_totals(df)`
- `weekly_totals = calculate_weekly_totals(df)`
- `build_summary = building_wise_summary(df)`

The building-wise summary is printed with clear headers so that each building’s average, minimum, maximum, and total consumption can be seen.

Task 3: Object-Oriented Modelling

To give a more realistic structure, three classes are created: `MeterReading`, `Building`, and `BuildingManager`.

- MeterReading stores timestamp and kwh for a single reading.
- Building has a name and a list of meter_readings and methods to add readings and calculate total consumption.
- BuildingManager maintains a dictionary of building objects and provides methods to add readings and generate reports for all buildings.

The script creates an instance of BuildingManager and iterates over df using df.iterrows(). For each row, manager.add_reading(row["building"], row["timestamp"], row["kwh"]) is called. At the end, manager.generate_all_reports() is used to print a textual summary for each building, showing total monthly consumption in kWh.

Task 4: Visualization using Matplotlib

Matplotlib is used to create a dashboard with three subplots. The figure size is set to (14, 10) for better clarity.

- Subplot 1 (Daily Trend): A line plot of daily_totals is drawn with dates on the x-axis and kWh on the y-axis, labelled as "Daily Electricity Usage".
- Subplot 2 (Weekly Bar Chart): A bar chart is created using weekly_totals with weeks on the x-axis and weekly kWh on the y-axis, labelled "Weekly Electricity Usage"
- Subplot 3 (Scatter Plot): A scatter plot is drawn using df["timestamp"] against df["kwh"] with alpha=0.5 to see the distribution of individual readings.

The layout is adjusted using plt.tight_layout(), and the figure is saved (recommended as "output/dashboard.png") and then closed.

Task 5: Exporting Clean Data and Summary

The script exports three main files into the "output" folder.

- cleaned_energy_data.csv: Contains all cleaned rows and the "source_file" column for reference.
- building_summary.csv: Contains building-wise statistics (mean, min, max, sum).
- summary.txt: A text report containing:
 - Title "Campus Energy Summary Report"
 - Total campus consumption using df["kwh"].sum() formatted to two decimals
 - Name of the highest consuming building using build_summary["sum"].idxmax()
 - Weekly totals and daily totals series printed in text form.

The console prints confirmation messages indicating that exports are complete and the project is finished.

RESULTS AND SCREENSHOTS

❖ Data Preview

- Admin Block

```
Python Lab 5 > data > admin_block_jan.csv > data
1 timestamp,kwh,building
2 2024-01-01,84.34,Admin_Block
3 2024-01-02,400.96,Admin_Block
4 2024-01-03,247.28,Admin_Block
5 2024-01-04,375.56,Admin_Block
6 2024-01-05,490.1,Admin_Block
7 2024-01-06,292.32,Admin_Block
8 2024-01-07,275.5,Admin_Block
9 2024-01-08,82.42,Admin_Block
10 2024-01-09,170.8,Admin_Block
11 2024-01-10,274.95,Admin_Block
12 2024-01-11,355.65,Admin_Block
13 2024-01-12,411.68,Admin_Block
14 2024-01-13,221.42,Admin_Block
15 2024-01-14,79.67,Admin_Block
16 2024-01-15,179.67,Admin_Block
17 2024-01-16,459.32,Admin_Block
18 2024-01-17,146.02,Admin_Block
19 2024-01-18,253.46,Admin_Block
20 2024-01-19,469.04,Admin_Block
21 2024-01-20,61.2,Admin_Block
22 2024-01-21,320.25,Admin_Block
23 2024-01-22,477.56,Admin_Block
24 2024-01-23,153.64,Admin_Block
25 2024-01-24,296.82,Admin_Block
26 2024-01-25,459.11,Admin_Block
27 2024-01-26,109.93,Admin_Block
28 2024-01-27,285.54,Admin_Block
29 2024-01-28,387.68,Admin_Block
30 2024-01-29,351.06,Admin_Block
31 2024-01-30,260.49,Admin_Block
32 2024-01-31,142.18,Admin_Block
33
```

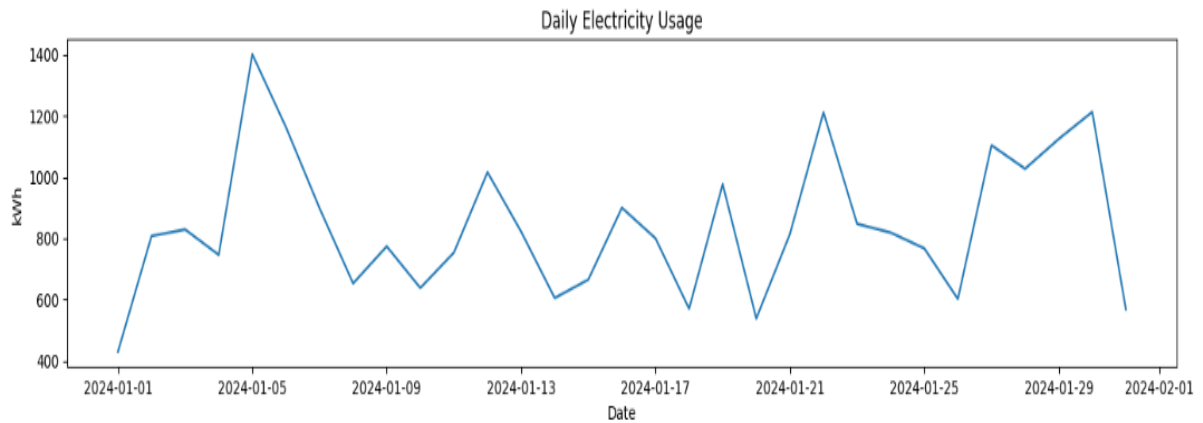
- Hostel

```
Python Lab 5 > data >  hostel_jan.csv >  data
1 timestamp,kwh,building
2 2024-01-01,73.91,Hostel
3 2024-01-02,188.98,Hostel
4 2024-01-03,316.67,Hostel
5 2024-01-04,155.8,Hostel
6 2024-01-05,484.24,Hostel
7 2024-01-06,475.27,Hostel
8 2024-01-07,431.78,Hostel
9 2024-01-08,262.55,Hostel
10 2024-01-09,428.66,Hostel
11 2024-01-10,109.0,Hostel
12 2024-01-11,188.93,Hostel
13 2024-01-12,258.35,Hostel
14 2024-01-13,383.83,Hostel
15 2024-01-14,268.62,Hostel
16 2024-01-15,111.59,Hostel
17 2024-01-16,204.59,Hostel
18 2024-01-17,195.99,Hostel
19 2024-01-18,185.19,Hostel
20 2024-01-19,124.48,Hostel
21 2024-01-20,236.71,Hostel
22 2024-01-21,251.65,Hostel
23 2024-01-22,398.71,Hostel
24 2024-01-23,408.38,Hostel
25 2024-01-24,285.08,Hostel
26 2024-01-25,257.28,Hostel
27 2024-01-26,400.2,Hostel
28 2024-01-27,449.28,Hostel
29 2024-01-28,353.71,Hostel
30 2024-01-29,410.22,Hostel
31 2024-01-30,472.6,Hostel
32 2024-01-31,68.3,Hostel
33
```

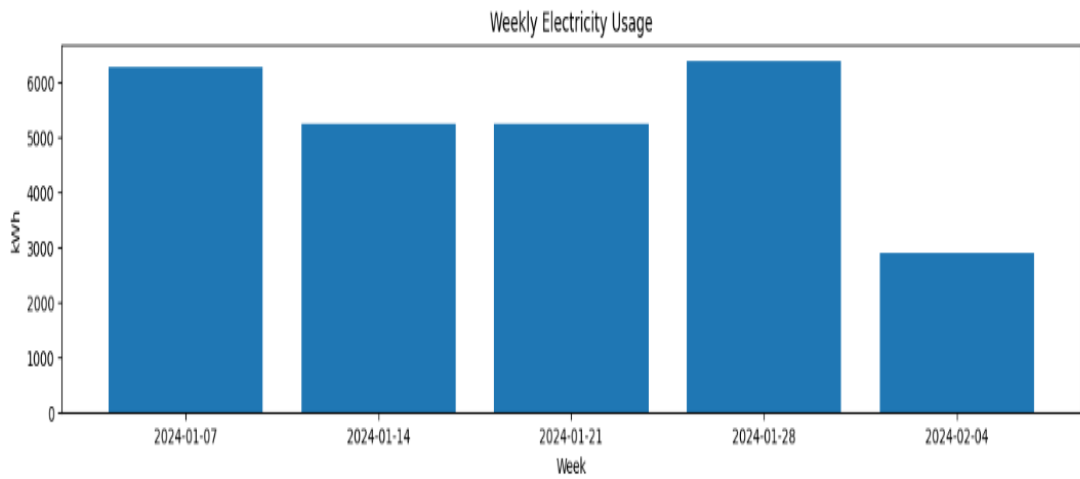
- Library

```
Python Lab 5 > data >  library_jan.csv >  data
1 timestamp,kwh,building
2 2024-01-01,270.84,Library
3 2024-01-02,217.57,Library
4 2024-01-03,264.83,Library
5 2024-01-04,214.65,Library
6 2024-01-05,427.06,Library
7 2024-01-06,395.89,Library
8 2024-01-07,191.3,Library
9 2024-01-08,307.68,Library
10 2024-01-09,174.22,Library
11 2024-01-10,253.78,Library
12 2024-01-11,208.84,Library
13 2024-01-12,345.83,Library
14 2024-01-13,216.66,Library
15 2024-01-14,256.59,Library
16 2024-01-15,373.7,Library
17 2024-01-16,235.85,Library
18 2024-01-17,457.89,Library
19 2024-01-18,131.2,Library
20 2024-01-19,383.5,Library
21 2024-01-20,240.07,Library
22 2024-01-21,241.9,Library
23 2024-01-22,335.47,Library
24 2024-01-23,285.31,Library
25 2024-01-24,236.7,Library
26 2024-01-25,50.64,Library
27 2024-01-26,91.52,Library
28 2024-01-27,369.23,Library
29 2024-01-28,285.96,Library
30 2024-01-29,363.27,Library
31 2024-01-30,479.96,Library
32 2024-01-31,357.31,Library
33
```

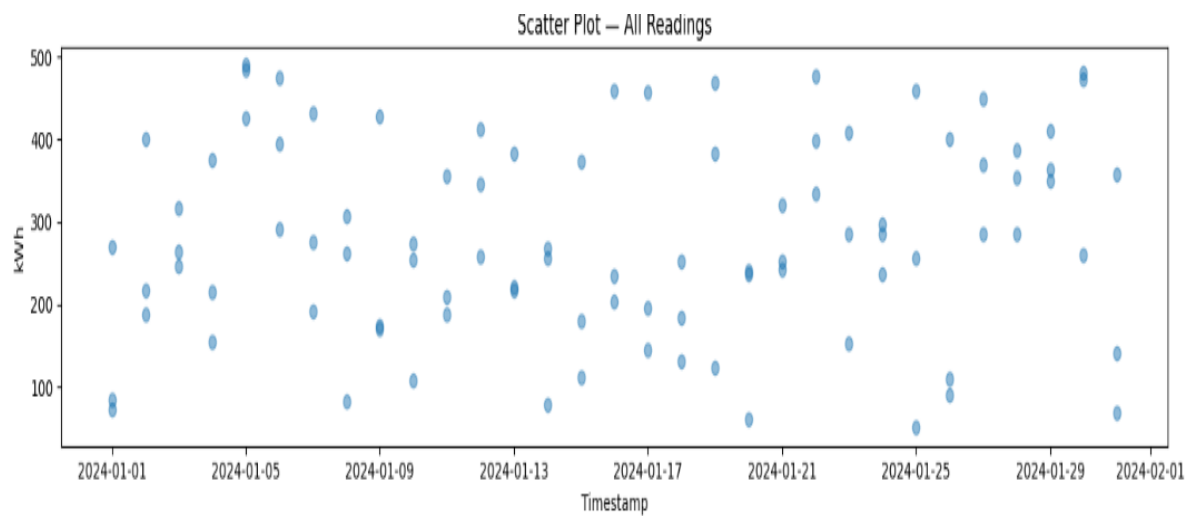
❖ Daily Electricity Usage



❖ Weekly Electricity Usage



❖ Scatter Plot (All Readings)



❖ Summary.txt

Python Lab 5 > output > summary.txt

```
1 Campus Energy Summary Report
2 -----
3 Total Campus Consumption: 26081.39 kWh
4 Highest Consuming Building: Hostel
5
6 Weekly Totals:
7 timestamp
8 2024-01-07      6274.85
9 2024-01-14      5260.13
10 2024-01-21      5263.27
11 2024-01-28      6377.75
12 2024-02-04      2905.39
13 Freq: W-SUN, Name: kwh, dtype: float64
14
15 Daily Totals:
16 timestamp
17 2024-01-01      429.09
18 2024-01-02      807.51
19 2024-01-03      828.78
20 2024-01-04      746.01
21 2024-01-05     1401.40
22 2024-01-06     1163.48
23 2024-01-07      898.58
24 2024-01-08      652.65
25 2024-01-09      773.68
26 2024-01-10      637.73
27 2024-01-11      753.42
28 2024-01-12     1015.86
29 2024-01-13      821.91
30 2024-01-14      604.88
31 2024-01-15      664.96
32 2024-01-16      899.76
33 2024-01-17      799.90
34 2024-01-18      569.85
35 2024-01-19      977.02
36 2024-01-20      537.98
37 2024-01-21      813.80
38 2024-01-22     1211.74
39 2024-01-23      847.33
40 2024-01-24      818.60
41 2024-01-25      767.03
42 2024-01-26      601.65
43 2024-01-27     1104.05
44 2024-01-28     1027.35
45 2024-01-29     1124.55
46 2024-01-30     1213.05
47 2024-01-31      567.79
48 Freq: D, Name: kwh, dtype: float64
```

❖ Cleaned energy data

```
Python Lab 5 > output > cleaned_energy_data.csv > data
1 timestamp,kwh,building,source_file
2 2024-01-01,84.34,Admin_Block,admin_block_jan.csv
3 2024-01-02,400.96,Admin_Block,admin_block_jan.csv
4 2024-01-03,247.28,Admin_Block,admin_block_jan.csv
5 2024-01-04,375.56,Admin_Block,admin_block_jan.csv
6 2024-01-05,490.1,Admin_Block,admin_block_jan.csv
7 2024-01-06,292.32,Admin_Block,admin_block_jan.csv
8 2024-01-07,275.5,Admin_Block,admin_block_jan.csv
9 2024-01-08,82.42,Admin_Block,admin_block_jan.csv
10 2024-01-09,170.8,Admin_Block,admin_block_jan.csv
11 2024-01-10,274.95,Admin_Block,admin_block_jan.csv
12 2024-01-11,355.65,Admin_Block,admin_block_jan.csv
13 2024-01-12,411.68,Admin_Block,admin_block_jan.csv
14 2024-01-13,221.42,Admin_Block,admin_block_jan.csv
15 2024-01-14,79.67,Admin_Block,admin_block_jan.csv
16 2024-01-15,179.67,Admin_Block,admin_block_jan.csv
17 2024-01-16,459.32,Admin_Block,admin_block_jan.csv
18 2024-01-17,146.02,Admin_Block,admin_block_jan.csv
19 2024-01-18,253.46,Admin_Block,admin_block_jan.csv
20 2024-01-19,469.04,Admin_Block,admin_block_jan.csv
21 2024-01-20,61.2,Admin_Block,admin_block_jan.csv
22 2024-01-21,320.25,Admin_Block,admin_block_jan.csv
23 2024-01-22,477.56,Admin_Block,admin_block_jan.csv
24 2024-01-23,153.64,Admin_Block,admin_block_jan.csv
25 2024-01-24,296.82,Admin_Block,admin_block_jan.csv
26 2024-01-25,459.11,Admin_Block,admin_block_jan.csv
27 2024-01-26,109.93,Admin_Block,admin_block_jan.csv
28 2024-01-27,285.54,Admin_Block,admin_block_jan.csv
29 2024-01-28,387.68,Admin_Block,admin_block_jan.csv
30 2024-01-29,351.06,Admin_Block,admin_block_jan.csv
31 2024-01-30,260.49,Admin_Block,admin_block_jan.csv
32 2024-01-31,142.18,Admin_Block,admin_block_jan.csv
33 2024-01-01,73.91,Hostel,hostel_jan.csv
34 2024-01-02,188.98,Hostel,hostel_jan.csv
35 2024-01-03,316.67,Hostel,hostel_jan.csv
36 2024-01-04,155.8,Hostel,hostel_jan.csv
37 2024-01-05,484.24,Hostel,hostel_jan.csv
38 2024-01-06,475.27,Hostel,hostel_jan.csv
39 2024-01-07,431.78,Hostel,hostel_jan.csv
40 2024-01-08,262.55,Hostel,hostel_jan.csv
41 2024-01-09,428.66,Hostel,hostel_jan.csv
42 2024-01-10,109.0,Hostel,hostel_jan.csv
43 2024-01-11,188.93,Hostel,hostel_jan.csv
44 2024-01-12,258.35,Hostel,hostel_jan.csv
45 2024-01-13,383.83,Hostel,hostel_jan.csv
```

❖ Building summary.csv

```
Python Lab 5 > output > building_summary.csv > data
1 building,mean,min,max,sum
2 Admin_Block,276.6329032258065,61.2,490.1,8575.62
3 Hostel,285.17903225806447,68.3,484.24,8840.55
4 Library,279.5232258064516,50.64,479.96,8665.22
5
```

GITHUB LINK: https://github.com/dev-2501730247/lab_assignment_krmu_Python_1-/tree/9f74cf92aec43138e7a236daa7b6821e35e5e6cf/lab5

DISCUSSION AND ANALYSIS

This chapter discusses the main patterns seen in the graphs and summaries and what they could mean for a typical campus. Even though the data is in CSV form and not live, the behaviour matches common building energy patterns like weekday peaks and building-wise differences.

The weekly bar chart gives a simpler view by combining daily readings into weekly totals. If one week has a much higher bar, it could be due to events, continuous lab usage, or seasonal effects like more AC usage in hot weeks. Weekly analysis is useful for planning budgets and comparing normal weeks with special or holiday week.

Building-wise and Scatter Plot Insights

The building-wise summary shows that some buildings consume much more electricity than others. High total (sum) values usually belong to hostels, main academic blocks, or labs, while smaller offices have lower totals. This helps the campus focus audits and energy-saving efforts on the top-consuming buildings first.

The scatter plot (timestamp vs kWh) gives a detailed view of individual readings. A dense cluster at a certain kWh level suggests a base load that is always on, while points far above the cluster may indicate unusual spikes or faults. If many points appear at night, it shows that some loads are running 24/7 and could be checked for optimisation.

Practical Use and Learning

Overall, combining daily, weekly, and building-wise analysis can support “green campus” efforts and smarter energy management. Management can use this type of report to spot waste, schedule maintenance on low-demand days, and monitor whether energy-saving steps are working over time.

From a learning point of view, this project shows how a simple Python script with Pandas and Matplotlib can convert raw CSV files into understandable insights. It also highlights that proper data cleaning (especially timestamps) and choosing the right plots (line, bar, scatter) are very important steps in any real data analysis.

ADVANTAGES AND APPLICATIONS

This project has several advantages, both from a technical point of view and from a practical campus usage point of view. Even though it is a small script, it shows how data analysis can directly support better energy decisions.

Advantages of the system:

- **Automated data handling:** The script automatically reads all CSV files from the data folder, merges them, and cleans them, so there is no need to manually open and combine files in Excel. This saves time for staff and reduces human error, especially when there are many files.

- **Error handling:** By using try–except while reading files and dropping invalid timestamps, the program continues to run even if one file is corrupted or has bad data. This makes the pipeline more reliable and closer to what is needed in real monitoring systems..
- **Time-series capable:** Using resampling for daily and weekly totals gives a proper time-series view instead of just raw rows. It becomes easy to switch between different time windows and spot patterns such as weekly cycles or seasonal trends.
- **Structured design:** The use of separate functions for aggregation and classes (MeterReading, Building, BuildingManager) makes the code easier to read, maintain, and extend. In the future, new methods (like cost calculations or alerts) can be added without rewriting everything..
- **Portable outputs:** The cleaned CSVs and summary.txt file can be shared with other departments, imported into spreadsheets, or even used in other software and dashboards. This makes the project useful beyond just one-time analysis

Possible applications

- **Regular energy reporting:** The script can be run every month or week to create quick reports for the administration, showing total consumption, top buildings, and changes compared to previous periods.
- **Identifying priority buildings:** Building-wise statistics help identify which buildings should be targeted first for energy audits, retrofits, or maintenance checks, such as replacing old lights or optimising AC usage
- **Supporting sustainability / “green campus” initiatives:** Energy data can back up sustainability goals by showing actual reductions (or increases) in usage over time when new policies are implemented. This makes planning more data-driven rather than based only on guesses
- **Educational use:** The project can be used in lab sessions to teach students about time-series analysis, Python programming, and real-world energy issues at the same time. Campuses often use their own buildings as living labs for data analytics and AI experiments

LIMITATIONS AND FUTURE SCOPE

Like any small student project, this system also has some limitations. At the same time, these limitations open up many ideas for future work and improvements.

Limitations:

- **Static CSV-based data only:** The project assumes that meter readings are already available in CSV files and does not cover how the data is collected in real time. In real campuses, data may come from smart meters, IoT sensors, or building management systems, which would need extra integration
- **Limited granularity and features:** The code focuses on daily and weekly totals and basic statistics like mean, min, max, and sum. It does not include

more detailed features such as hourly profiles, occupancy data, or weather conditions, which are important for deeper energy analysis.

- **Basic visualisation only:** The visualisations are static PNG images created with Matplotlib. There is no interactive dashboard where users can zoom, filter by building, or change time ranges on the fly, which modern energy monitoring systems usually provide.

Future Scope

- **Real-time integration with IoT and smart meters:** In future, the CSV reading part can be replaced or complemented by APIs or MQTT feeds that pull live data from smart meters and IoT devices. This would allow near real-time monitoring of the campus and faster reaction to unusual energy use. Add monthly and yearly summaries and comparisons.
- **More detailed time resolutions and features:** The system can be extended to analyse hourly or even 15-minute interval data, which is common in smart meter systems. Extra features such as outdoor temperature, classroom schedules, or occupancy can be merged to understand why certain peaks occur.
- **Machine learning and forecasting:** By using the cleaned data as a base, machine learning models like Random Forest, XGBoost, or Support Vector Regression can be applied to forecast future energy demand for each building or the entire campus. These forecasts could help plan capacity, schedule loads, and negotiate better tariffs.
- **Web-based interactive dashboard:** A web application using Flask, Django, or a dashboard tool can be built on top of this analysis to provide interactive charts, filters, and drill-downs per building or time period. Users like administrators and facility managers could access it in a browser instead of reading text files.
- **Cost and emissions estimation:** By combining kWh data with tariff information and emission factors, the system can also estimate monthly electricity bills and approximate CO₂ emissions. This would make the reports even more meaningful in terms of both money and environmental impact.

REFLECTION

Working on this project was a good learning experience because it connected theory from programming and data analysis classes with a practical problem like campus energy consumption. At the beginning, even simple things like handling multiple CSV files, managing paths, and converting timestamps felt a bit confusing, but step by step it became clearer as the workflow was structured properly. This made me realise how important it is to follow a proper data analysis pipeline instead of jumping directly to plotting graphs.

Using Pandas and Matplotlib together also helped to build more confidence in Python skills. Debugging errors like wrong column names, NaN timestamps, and plotting issues forced me to carefully read documentation and try small experiments, which improved my problem-solving approach. The OOP part with Building and MeterReading classes was another area where I could see how object-oriented concepts from class can actually make real code more organised and reusable.

Overall, this mini-project showed that even basic energy data, when cleaned and visualised properly, can give meaningful insights that are useful for a campus. It also motivated me to explore more advanced topics in the future, such as using machine learning for energy forecasting and integrating live IoT data from smart meters.

CONCLUSION

This project successfully demonstrates how Python and Pandas can be used to analyze campus energy consumption data in an organized manner. The system reads multiple CSV files, cleans the data, performs daily and weekly aggregations, generates building-wise summaries, and exports results for further use.

Visualizations created with Matplotlib provide a clear picture of how energy usage changes over time and which buildings contribute most to the total consumption.

Working on this project improved understanding of time-series data, data cleaning, resampling, grouping, and plotting. It also gave hands-on experience with object-oriented programming in Python by modeling buildings and meter readings as separate classes. Overall, this mini-project can act as a foundation for more advanced energy management systems and smart campus solutions in the future.

REFERENCES

- Online resources on Python projects from **w3school.com** for energy data analysis and Pandas usage.
- Articles on benefits of building energy monitoring systems.
- General guidelines for engineering project report formatting.